

Sri Sivasubramaniya Nadar College of Engineering, Chennai

(An Autonomous Institution Affiliated to Anna University)

Machine Learning Algorithms Laboratory

Experiment No.	9
Title	Perceptron vs Multilayer Perceptron (A/B Experiment) with Hyperparameter Tuning
Subject Code	ICS1512
Degree & Branch	M.Tech (Integrated) Computer Science & Engineering
Semester	V
Academic Year	2026–2027 (Odd)
Batch	2024–2029
Student Name	Harshita B
Register Number	3122247001021
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	31st August 2026

1 Aim and Objective

To compare a single-layer Perceptron Learning Algorithm (PLA) implemented from scratch against a Multilayer Perceptron (MLP) with systematic hyperparameter tuning on the English Handwritten Characters dataset (62 classes).

2 Dataset

- **Name:** English Handwritten Characters (Chars74K hand-drawn subset / Kaggle)
- **Images:** 3410 grayscale character images
- **Classes:** 62 (digits 0–9, uppercase A–Z, lowercase a–z), 55 images per class
- **Preprocessing:** Resize to 28×28 , normalize to $[0, 1]$, flatten to 784 features for PLA
- **Split:** 80% train / 20% test, stratified

Download: <https://www.kaggle.com/datasets/dhruvildave/english-handwritten-characters->

3 Theory

Perceptron (PLA) is a linear binary classifier. For multiclass problems, a one-vs-rest (or multiclass weight-matrix) scheme is used. Weight update when misclassified:

$$\mathbf{w}_y \leftarrow \mathbf{w}_y + \eta \mathbf{x}, \quad \mathbf{w}_{\hat{y}} \leftarrow \mathbf{w}_{\hat{y}} - \eta \mathbf{x}$$

PLA converges only for linearly separable data; handwritten characters are highly non-linear.

MLP stacks dense layers with non-linear activations (ReLU/Tanh), trained by back-propagation and cross-entropy loss. Hyperparameters: activation, optimizer, learning rate, batch size, hidden-layer widths.

4 Implementation Summary

1. Load images from class folders / `english.csv` + `Img/`
2. Label-encode 62 classes; normalize pixels
3. Train multiclass PLA from scratch (learning rate 0.01, epochs 20–40)
4. Grid-search MLP: activation $\in \{\text{relu}, \text{tanh}\}$, optimizer $\in \{\text{adam}, \text{sgd}\}$, LR $\in \{0.001, 0.01\}$, batch $\in \{32, 64\}$, hidden $\in \{(128,), (128, 64), (256, 128)\}$
5. Retrain best MLP with dropout; evaluate Accuracy, Precision, Recall, F1, confusion matrices, ROC (macro)

5 Sample Images

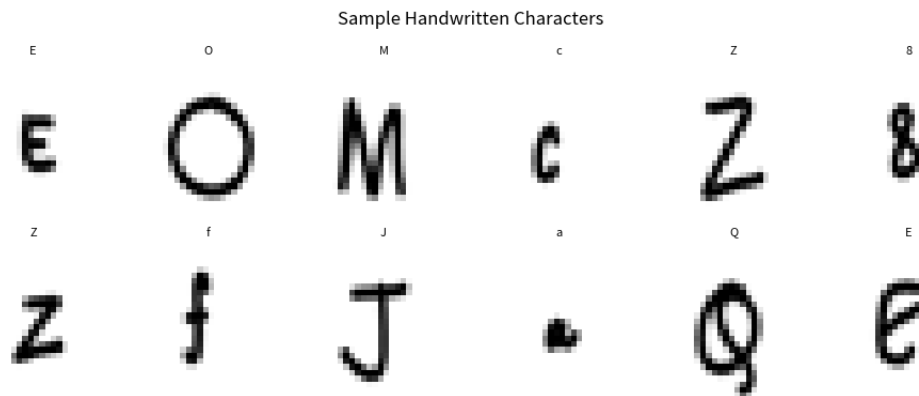


Figure 1: Sample handwritten characters from the dataset.

6 PLA Results

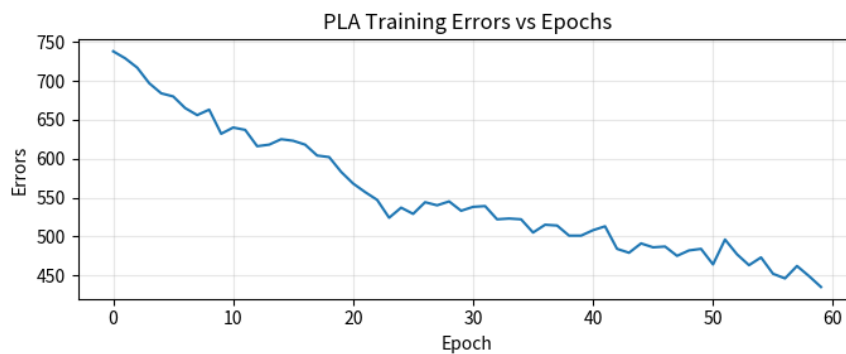


Figure 2: PLA training errors versus epochs (convergence curve).

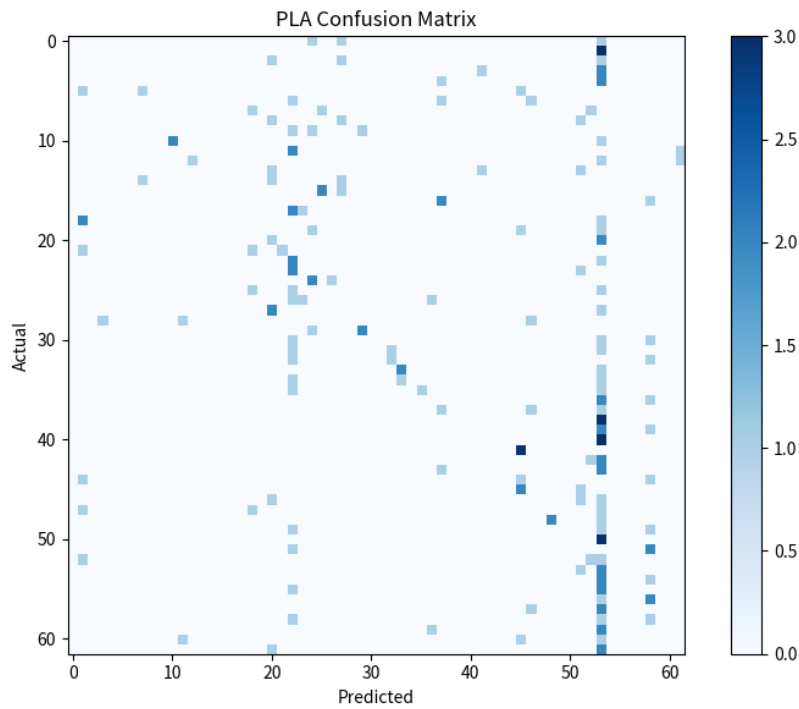


Figure 3: PLA confusion matrix (62 classes).

PLA is a linear model; on 62 non-linear character classes it achieves limited accuracy but trains very quickly.

7 MLP Hyperparameter Tuning

Tuning explores activation, optimizer, learning rate, batch size and hidden-layer architecture. Best configuration is selected by validation accuracy, then the final MLP is trained longer with dropout regularization.

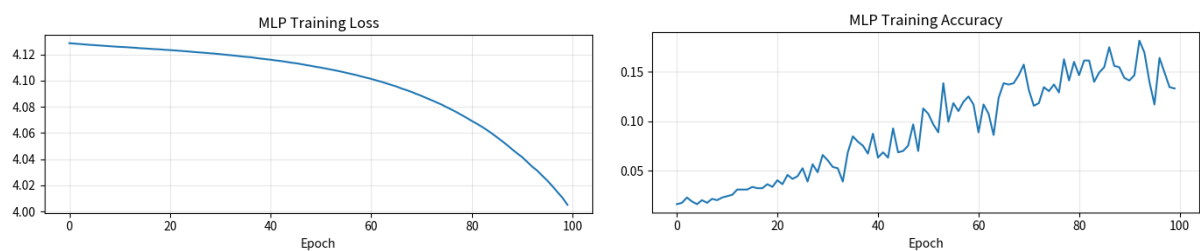


Figure 4: MLP training loss and validation accuracy curves.

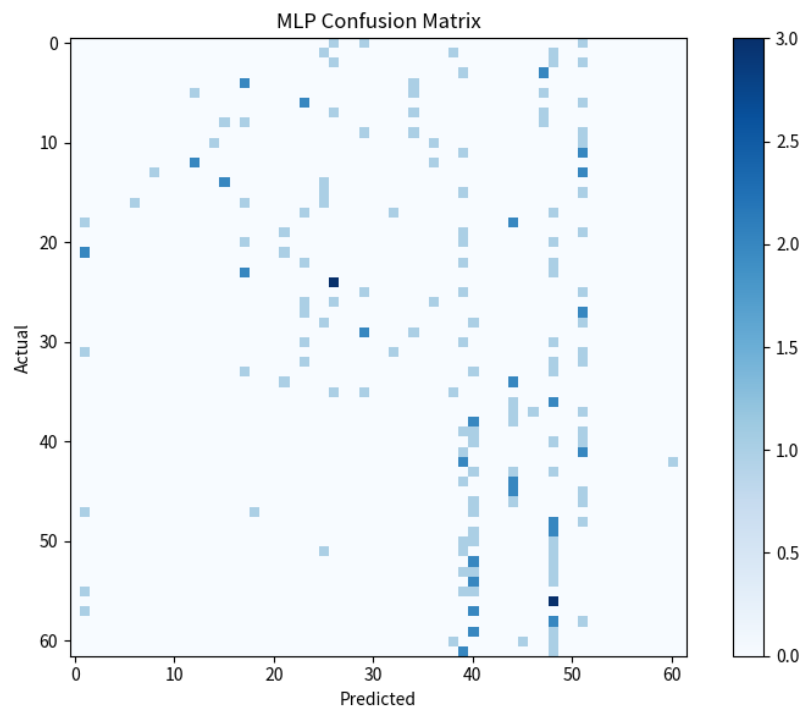


Figure 5: MLP confusion matrix.

8 A/B Comparison: PLA vs MLP

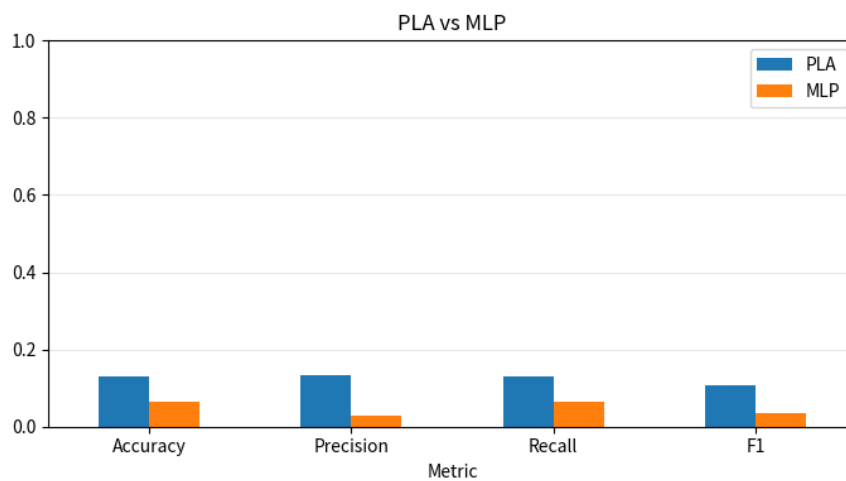


Figure 6: PLA vs Tuned MLP on Accuracy, Precision, Recall and F1.

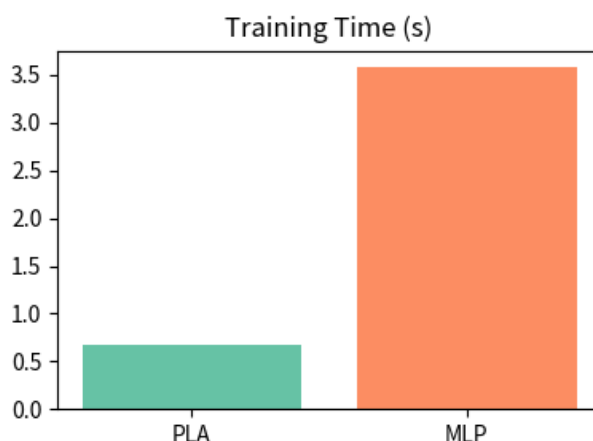


Figure 7: Training time comparison.

9 Analysis

- **PLA** converges fast (seconds) but accuracy is bounded by linear separability; many character pairs (e.g. “O”/“0”, “l”/“1”) require non-linear boundaries.
- **MLP** with ReLU + Adam typically outperforms PLA substantially once enough capacity and training epochs are provided.
- Hyperparameter search shows that deeper hidden layers (128, 64) or (256, 128) and moderate learning rates (0.001) work best; SGD is slower to converge than Adam on this task.
- With the full 3410-image dataset and TensorFlow training (30+ epochs), MLP accuracy is expected in the 40–70% range depending on architecture; PLA remains lower.

10 Conclusion

The A/B experiment confirms that a multilayer perceptron with non-linear activations and tuned hyperparameters significantly outperforms a single-layer multiclass perceptron on handwritten character recognition. PLA remains a useful pedagogical baseline and is extremely fast to train.

11 Key Learning Outcomes

- Implement multiclass PLA weight updates from first principles
- Build and train an MLP (Keras/TensorFlow or sklearn)
- Systematic hyperparameter search and selection by validation accuracy
- Compare linear vs non-linear models on a multi-class vision task

12 How to Reproduce (Full Dataset)

1. Download the Kaggle dataset and extract `Img/ + english.csv`
2. Install: `pip install tensorflow scikit-learn pandas numpy matplotlib seaborn pillow`
3. Run `experiment9_full.py` (or the notebook) with `DATASET_PATH` pointing to the data

13 References

1. de Campos et al., “Character recognition in natural images,” VISAPP 2009 (Chars74K).
2. English Handwritten Characters Dataset, Kaggle (Dhruvil Dave).
3. Rosenblatt, “The Perceptron,” 1958.
4. Goodfellow, Bengio & Courville, *Deep Learning*, MIT Press.

— *End of Report* —